

start.io | February 2026

THE CLAUDE BEAST

LSP: Language Server Protocol

Semantic Code Intelligence for AI-Assisted Development

What is LSP?

A standardized protocol between editors and language servers, created by Microsoft in 2016.

Before LSP

- Every editor needed its own implementation for each language
- Combinatorial explosion

After LSP

- One language server works in VS Code, Neovim, and Claude Code
- Decoupled architecture
- Universal code intelligence

Why LSP Matters for AI Coding Agents



Without LSP (grep-based)

- Returns false positives (comments, strings, similar names)
- Takes seconds scanning raw files
- Cannot distinguish local vs. module-level variables
- Burns excessive tokens processing irrelevant matches



With LSP (semantic)

- Type information & documentation inline
- Instant jump-to-definition
- Scope-aware reference finding
- Auto-diagnostics after every edit
- self-correcting errors

9 Core LSP Operations

Claude Code exposes these standard LSP commands for semantic code navigation

1 **goToDefinition**

Jump to where a symbol is defined

2 **findReferences**

Locate all usages of a symbol

3 **hover**

Get type signatures & docs inline

4 **goToImplementation**

Navigate interface to concrete class

5 **documentSymbol**

View hierarchical file structure

6 **workspaceSymbol**

Search symbols across workspace

7 **prepareCallHierarchy**

Initialize call hierarchy analysis

8 **incomingCalls**

Find all callers of a function

9 **outgoingCalls**

Find all functions a target calls

TypeScript LSP Setup

1 Enable the LSP Tool

```
export ENABLE_LSP_TOOL=1    or add to  
~/.claude/settings.json
```

2 Install Language Server

```
npm install -g typescript-language-server  
typescript
```

3 Install the Plugin

```
/plugin install typescript-lsp@claude-  
plugins-official
```

4 Restart Claude Code

LSP requires a fresh session to initialize properly



Verify It Works

Ask: "Go to the definition of the main function"

Working: "The function main is defined in src/index.ts at line 42"

Not working: "I'm searching through the codebase for main..."

Multi-Language Support

11 Official Anthropic Languages

- ▶ TypeScript / JavaScript
- ▶ Go (gopls)
- ▶ Java (jdtls)
- ▶ C# (csharp-ls)
- ▶ Kotlin
- ▶ PHP (intelephense)
- ▶ Python (Pylint)
- ▶ Rust (rust-analyzer)
- ▶ C/C++ (clangd)
- ▶ Swift (sourcekit-lsp)
- ▶ Lua

17+ Community Languages

`Piebald-AI/claude-code-lsps`

- ▶ All 11 official + PowerShell
- ▶ HTML / CSS
- ▶ LaTeX (texlab)
- ▶ Julia
- ▶ Vue (v2.x)
- ▶ OCaml
- ▶ BSL / 1C:Enterprise



Multi-language projects:

Install plugins for each language. Claude Code automatically activates the right server based on file extensions.

The Three Iron Laws of LSP Usage



LAW 1

No Modifying Unfamiliar Code Without `goToDefinition` First

Always understand implementation before changing it. Prevents assumptions based on function names.



LAW 2

No Refactoring Without `findReferences` Impact Analysis

Always assess the full scope of changes.
Prevents the 'quick refactor' that misses half the call sites.



LAW 3

No Claiming Code Works Without LSP Diagnostics

Always check for type errors after modifications. Catches mismatches and missing imports immediately.

Setup & Configuration

Settings File Locations

- ▶ **User (global):** `~/.claude/settings.json`
Personal plugins, all projects
- ▶ **Project (shared):** `.claude/settings.json`
Team plugins via version control
- ▶ **Local (private):**
`.claude/settings.local.json`
Project-specific, gitignored

Plugin Configuration (.lsp.json)

- ▶ **"command"** Language server binary (required)
- ▶ **"extensionToLanguage"** File ext mapping (required)
- ▶ **"args"** CLI arguments for the server
- ▶ **"transport"** stdio (default) or socket
- ▶ **"restartOnCrash"** Auto-restart on failure

Quick Troubleshooting

"No LSP server available"

→ Install plugin + restart Claude Code

"Executable not found"

→ Install the binary (npm/pip/brew)

Plugin installed but silent

→ Clear cache: `rm -rf ~/.claude/plugins/cache`

Diagnostics not updating

→ Check /plugin Errors tab, force refresh

Key Takeaways

- LSP transforms Claude Code from a text processor into a tool with genuine semantic code understanding
 - 900x faster navigation, 75% fewer tokens, and elimination of false positives
 - 11 official + 17+ community languages supported with 5-minute setup per language
 - The Three Iron Laws enforce best practices:
Navigate, Analyze, Inspect, then Edit
 - 200K context window + LSP semantic understanding
- = uniquely powerful AI development



Adoption Path

1. Enable LSP for primary language
2. Verify with `goToDefinition` test
3. Add plugins for other languages
4. Consider lsp-tools for best practices
5. Share settings via `.claude/settings.json`